

Maestría en Software

Autora: Ing. Patsy Malena Prieto MSc.



| POSGRADOS |

Bienvenida

UNIDAD 4 - Desarrollo de sistemas basados en APIs

Tema:

- Autorización y autenticación
- Implementación de servicios
- Pruebas unitarias
- Pruebas de stress
- Despliegue de APIs

El Ciclo de Vida de una API de Grado de Producción



1. Seguridad

Autenticación,
Autorización y
JSON Web Tokens.



2. Implementación

Principios RESTful,
OpenAPI vs. gRPC,
Versionamiento.



3. Testing

Cuadrantes,
Pirámides y
Contratos.



4. Despliegue

Infraestructura
de Tráfico y
Estrategias de
Liberación.

Autenticación vs. Autorización

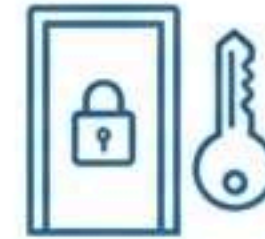


Autenticación

Quién eres

El proceso por el cual un usuario o dispositivo se identifica de forma inequívoca.

Es el primer paso; puede utilizarse como factor para decisiones posteriores.



Autorización

Qué puedes hacer

El acto de permitir o denegar a los usuarios y dispositivos los derechos de acceso a determinados recursos de la red.

Modelos de Control de Acceso

Autenticación (Mecanismos)

Básica (Basic Auth)

Envío de credenciales codificadas en Base64. Insegura si no se emplea HTTPS.

Token-Based (JWT)

Tokens firmados digitalmente para sesiones sin estado (stateless).

Federada (OAuth2 / OIDC)

Delegación de identidad hacia proveedores externos (ej. Google, Microsoft).

Autorización (Aplicación)

Roles (RBAC)

Conjunto de permisos agrupados por perfil predefinido.

Scopes

Límites de acceso específicos para operaciones (ej. lectura, escritura, admin).

Políticas (ABAC)

Reglas dinámicas basadas en contexto o atributos del usuario o entorno.

Anatomía de un JSON Web Token (JWT)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 • eyJzdWliOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaGgRG9IiwiaWF0IjoxNTE2MjM5MDIyfQ • SflKxwR3MeKkf2Q7T4fwpMeJf36P0k6yJV_adQssw5c

Header

Contiene el tipo de token ("typ": "JWT") y el algoritmo de firma/cifrado utilizado (ej. HS256).

Payload (Claims)

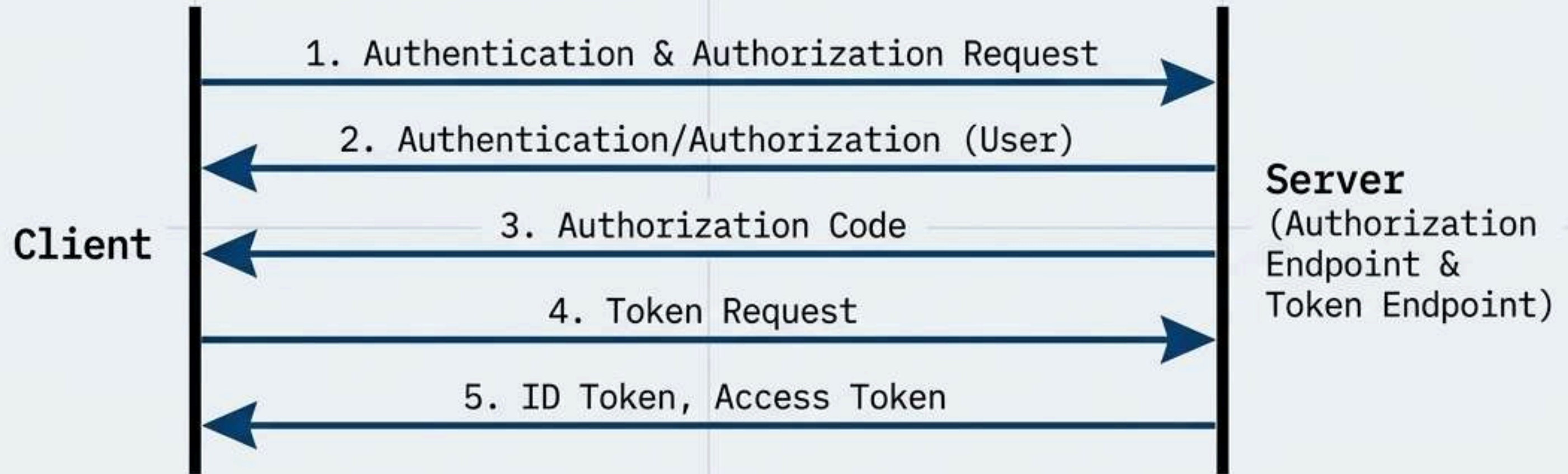
Datos que identifican al usuario. Se dividen en: Registrados (estándar IANA), Públicos (definidos sin colisiones) y Privados (datos propios de la app).

Signature

Un hash generado a partir del header y payload, diseñado para validar la autenticidad y prevenir alteraciones.

OpenID Connect (OIDC) y Delegación de Identidad

OIDC es una capa de identidad sobre OAuth 2.0. Utiliza un esquema REST y JSON para verificar identidades y obtener perfiles.



Regla Crítica de Arquitectura: OIDC proporciona identidad, pero no acceso directo. Los ID Tokens (claims sobre el usuario) NUNCA deben utilizarse como sustitutos de los Access Tokens para acceder a recursos protegidos.

Principios de Implementación RESTful

Fundamentos

Características Core

Un servicio API debe ser: Idempotente, Predecible y Autodescriptivo.

El enfoque debe ser "Contract-First" (diseño antes de implementación).

Validación rigurosa de entradas y salidas para integridad de datos.

Modelado de Recursos

Nouns, Not Verbs

Regla: Usar sustantivos (recursos), no verbos (acciones). Siempre en plural.

Do / Don't

✗ POST /crearPaciente

✓ POST /pacientes

✗ GET /obtenerCita/123

✓ GET /citas/123

Versionamiento Semántico (SemVer)

El versionamiento asegura durabilidad y maneja desafíos de compatibilidad para múltiples consumidores en producción.

Major



Cambio No Compatible. Rompe la compatibilidad anterior. Requiere acción del consumidor (ej. guía de migración).

v2.1.4

Patch



Corrección de Errores. Cambios internos exclusivos para solucionar bugs en la funcionalidad existente.

Minor



Cambio Aditivo. Introduce nuevas características de forma retrocompatible (*backward compatible*). El cliente no necesita cambiar código para seguir funcionando.

Cheat Sheet: Buenas Prácticas de Implementación

Categoría	Recomendación
Rutas	Usar nombres claros y plurales (/pacientes).
Respuestas	Utilizar códigos HTTP estándar (200, 201, 404, 500).
Errores	Manejar excepciones de forma centralizada (middleware).
Paginación	Implementar parámetros estandarizados (?page y ?limit).
Logs	Registrar operaciones CRUD, errores y tiempos de respuesta.
Seguridad	Validar el JWT y aplicar scopes granulares por recurso.
Estilo	Documentar rigurosamente con OpenAPI para mantener consistencia semántica.

Especificación de Contratos: OpenAPI vs. gRPC



OpenAPI Specification (OAS)

Mecanismo estándar para describir APIs REST.

Basado en formatos legibles por humanos: JSON o YAML.

Transmite metadatos, seguridad, y ejemplos (reemplazó a Swagger).

Restricciones Tolerantes: Añadir campos adicionales generalmente no rompe la compatibilidad con clientes existentes.



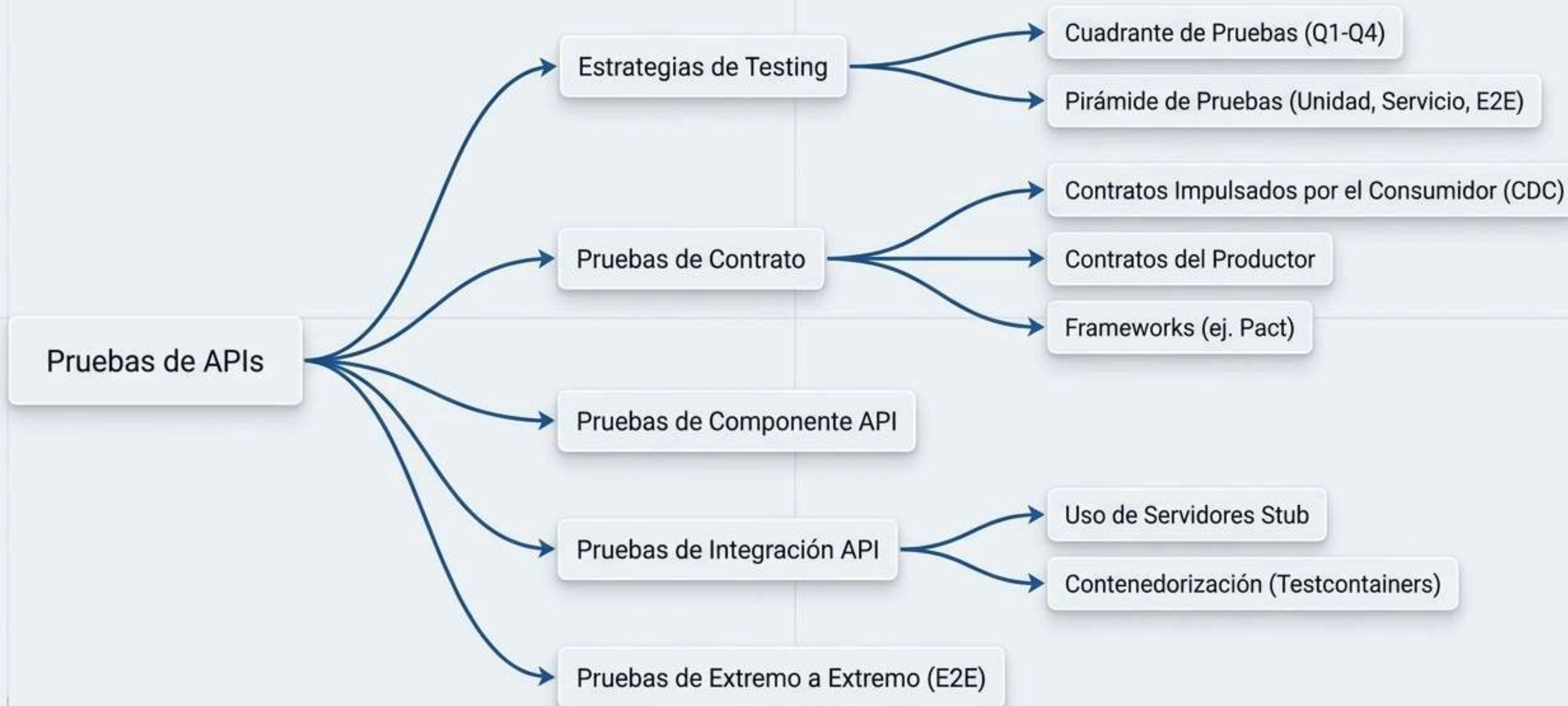
gRPC (Remote Procedure Call)

Diseñado para APIs de comunicación de alto rendimiento.

Basado en esquemas estrictos mediante archivos .proto y diseño de mensaje binario.

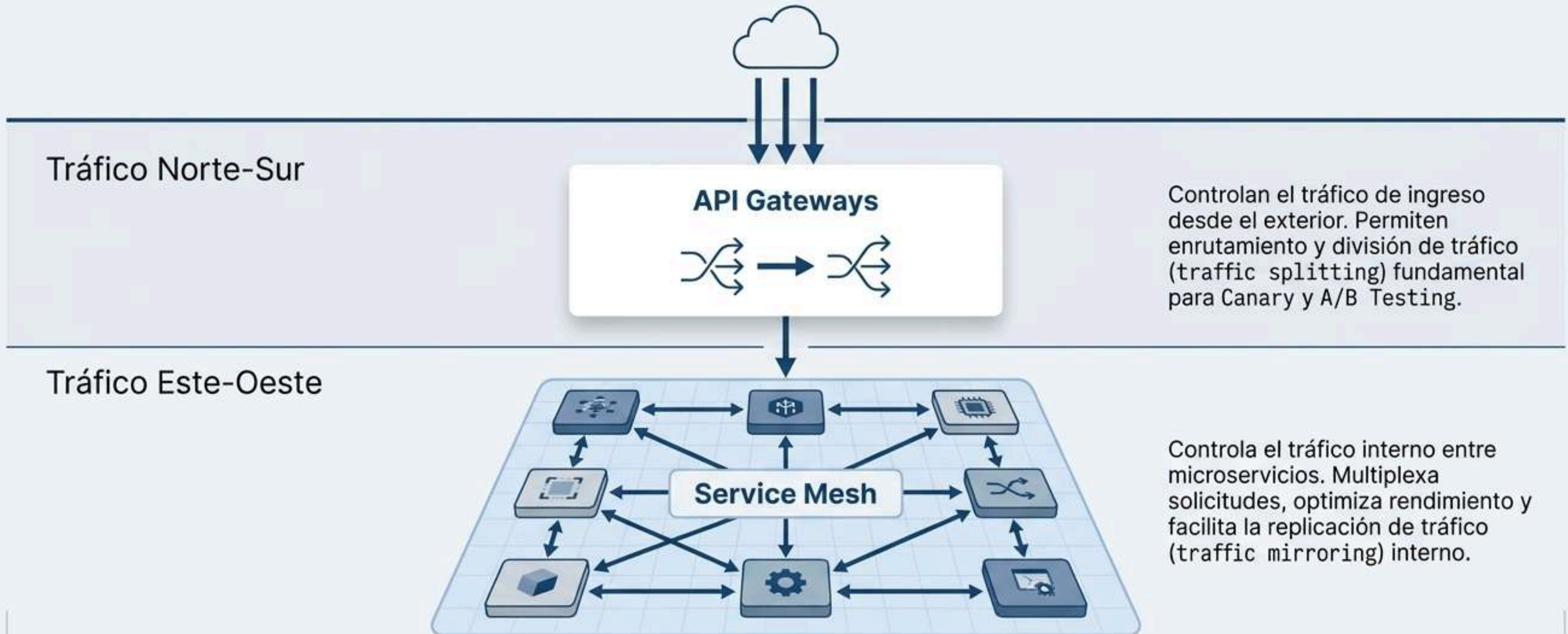
Restricciones Críticas: Cambiar tipos de datos, orden, eliminar o renombrar campos romperá la compatibilidad inmediatamente. El versionado es inflexible.

El Ecosistema de Pruebas de APIs



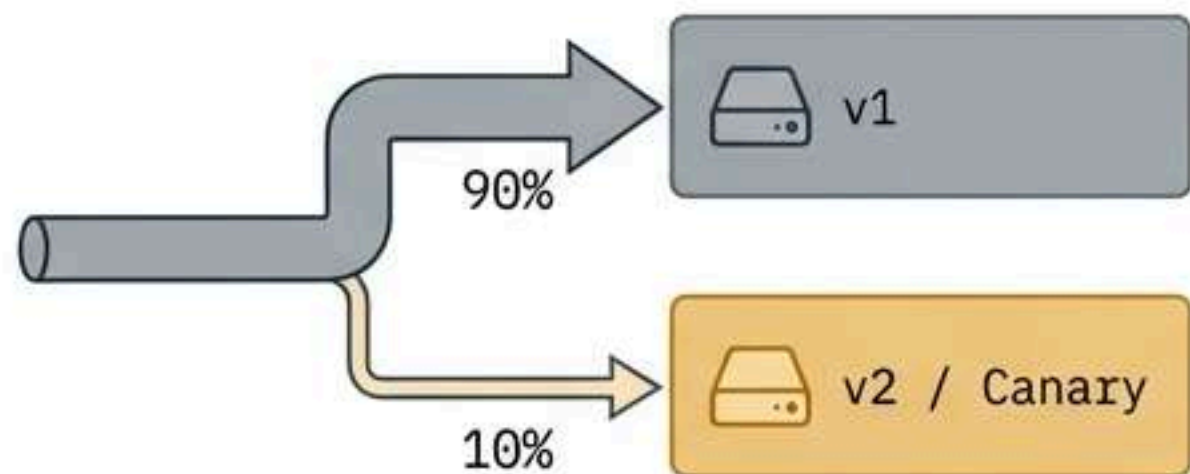
Arquitectura de Tráfico en Entornos Containerizados

El despliegue en Kubernetes se automatiza en el pipeline. Para permitir liberaciones avanzadas, comprender la topología de red es vital.



Estrategias de Liberación: Mitigación de Riesgos

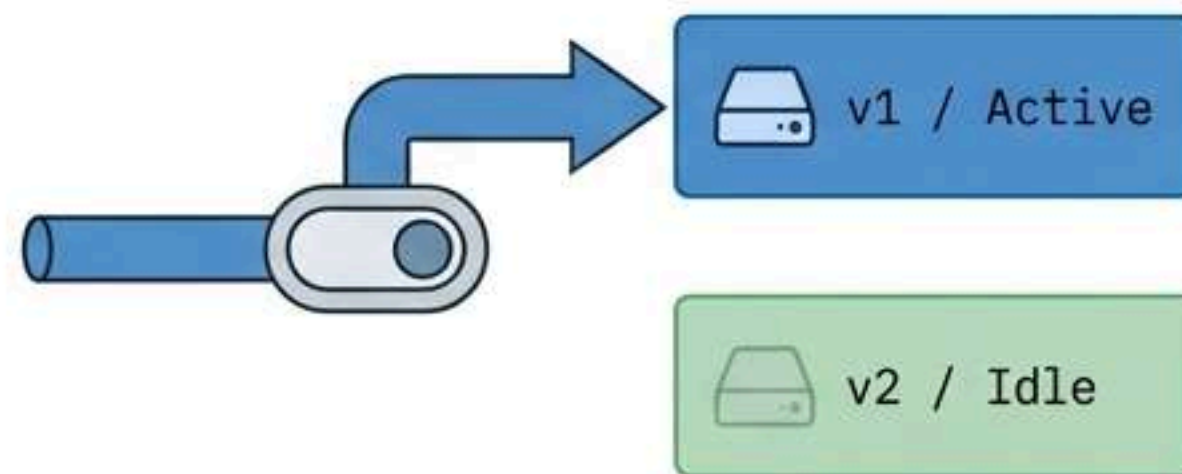
Canary Releases



Mechanism: Introduce la nueva versión a un pequeño porcentaje del tráfico real.

Use Case: Ideal para monitorear métricas técnicas (latencia, errores) y KPIs de negocio con bajo riesgo. Promoción gradual y automatizada.

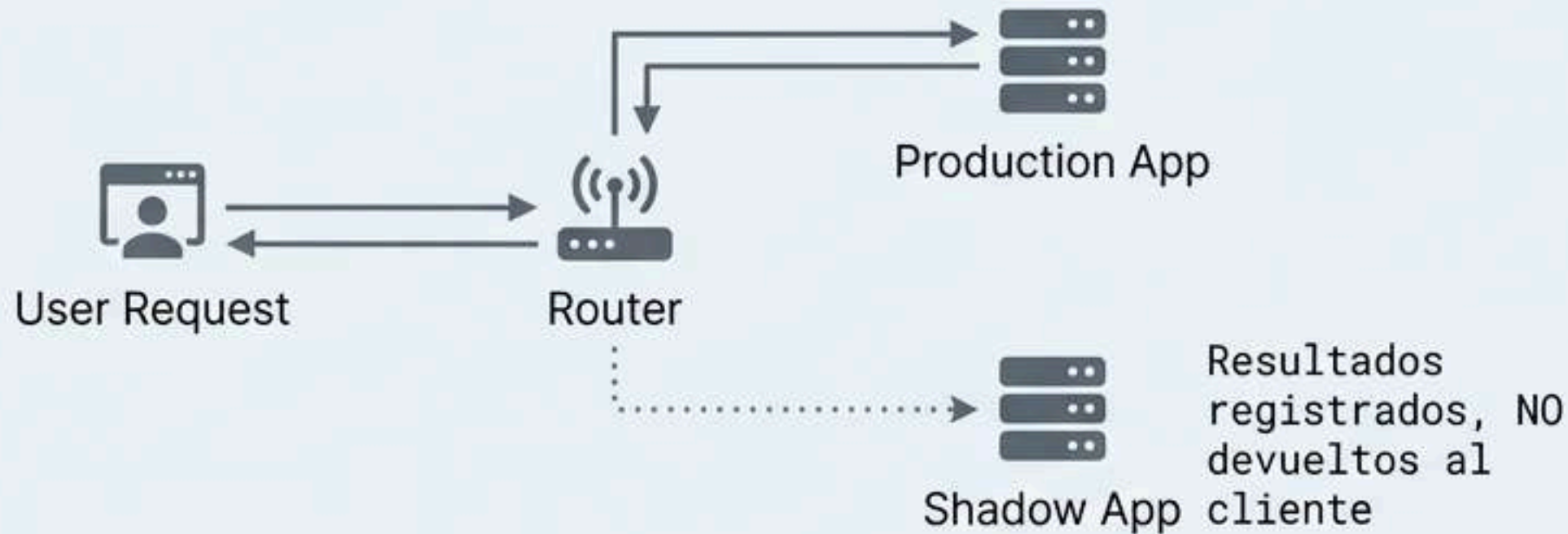
Blue-Green Deployment



Mechanism: Despliegue de un entorno 'verde' inactivo en paralelo al 'azul' activo. Conmutación (flip) total del tráfico en un instante.

Use Case: Requiere el doble de infraestructura. Permite una reversión (rollback) inmediata si se detectan problemas.

Traffic Mirroring y la Separación de Preocupaciones



Se copia o duplica el tráfico de producción hacia una ubicación adicional.
Permite lanzamientos **"en la oscuridad"** (out-of-band) para evaluar el rendimiento **operativo y estrés real** antes de **afectar a un solo usuario**.

Desplegar $\neq$ Liberar.

La clave de las arquitecturas API modernas es separar el Despliegue (instalar el código en los servidores de forma segura) de la Liberación (exponer esas nuevas características al tráfico del usuario mediante el Gateway o Mesh).

Bibliografía

- Munonye, K., Péter, M. Machine learning approach to vulnerability detection in OAuth 2.0 authentication and authorization flow. Int. J. Inf. Secur. 21, 223–237 (2022).
<https://doi.org/10.1007/s10207-021-00551-w>
- Oh S-R, Kim Y-G. AFaaS: Authorization framework as a service for Internet of Things based on interoperable OAuth. International Journal of Distributed Sensor Networks. 2020;16(2).
[doi:10.1177/1550147720906388](https://doi.org/10.1177/1550147720906388)
- Siriwardena, P. (2020). Advanced API security: OAuth 2.0 and beyond. Apress.